

Literature Survey — Model 3

On-Device Recommender with Periodic Private Updates

Scope

Model 3 makes three claims: that a recommender can be made small enough to run on a phone, that it can adapt to an individual user, and that the information it sends back can be made private. The three papers below are the published work behind each of those claims. Each is reviewed for what it did, what it found, where it falls short, and what it means for our design.

SASRec (Kang & McAuley, ICDM 2018) is covered separately as the project baseline and is not repeated here.

Paper 1 — Learning Elastic Embeddings for Customizing On-Device Recommenders

Chen, Yin, Zheng, Huang, Wang & Wang. **KDD 2021**. DOI 10.1145/3447548.3467220

Summary. The closest published work to what Model 3 is trying to do. The authors observe that existing memory-efficient recommenders are each built for **one fixed memory budget**, so moving to a smaller or larger device means retraining the model from scratch. They call this a “train once, get one” scheme and propose an alternative that trains once and can then be fitted to any budget without retraining.

Methods and models used. Their system is called RULE, built on top of LightGCN. Instead of giving every item one embedding vector of fixed length, they learn a pool of **embedding blocks**, and each item’s representation is a concatenation of whichever blocks are chosen for it. Two mechanisms make this work. A **diversity-driven regularisation** term during training forces the blocks to be different from one another, so none are wasted. Then an **evolutionary search guided by a performance estimator** picks, for a given memory budget, which blocks each item should use. Because the search runs after training, one trained model can be specialised for many budgets.

Results. Evaluated on Amazon-Book (52,643 users, 91,599 items, 2.98M interactions, 99.94% sparse) and Yelp2020 (138,322 users, 105,843 items, 3.87M interactions, 99.97% sparse), with users under 10 interactions and items under 10 users filtered out, split 70/10/20. Reported at three memory budgets, **5 MB, 10 MB and 25 MB**, using Recall@K and NDCG@K for K = 50 and 100.

RULE beat all six comparators at every budget, and the margin was largest at the tightest budget. On Amazon-Book at 5 MB it reached Recall@50 of 0.0534 against 0.0507 for LightGCN and 0.0456 for LightRec. The comparators span three families: fixed-size embeddings (PMF, LightGCN), AutoML dimension search (ESAPN, AutoEmb), and compositional embeddings (DLRM-CE, LightRec). Notably the two AutoML methods performed **worse** than simpler approaches under tight budgets, which the authors attribute partly to their being designed for rating prediction rather than ranking.

They also measured inference time in two device settings, described as **IoT (single-core CPU)** and **Cellphone (four-core CPU)**, and report keeping inference under 1000 ms.

Limitations. The evolutionary search is an extra offline cost, and the paper's own ablations show the result depends on hyperparameters of the search (the number of item groups G and blocks per item N). The method is built on LightGCN, a graph-based collaborative filtering model, not a sequential recommender, so it does not directly transfer to our architecture. Privacy is given as the *motivation* for going on-device but is never measured — there is no privacy budget and no attack.

Relevance to Model 3. Three things we take from it. First, it **validates our measurement approach**: a KDD paper treats a constrained CPU as the on-device setting rather than using real phones, which is exactly what the professor has asked us to do. Second, its memory budgets of 5, 10 and 25 MB tell us what counts as a realistic target, and confirm that our MovieLens model at roughly 0.8 MB is far below the range where this problem is interesting — supporting the case for a larger-catalogue dataset. Third, it is the strongest comparator we could cite for our own compression results, though reimplementing it is out of scope.

Paper 2 — Compositional Embeddings Using Complementary Partitions for Memory-Efficient Recommendation Systems

Shi, Mudigere, Naumov & Tran. **KDD 2020**. DOI 10.1145/3394486.3403059

Summary. Addresses the same bottleneck from a different angle. In industrial recommenders the embedding tables can require multiple gigabytes each, and the standard fix is the hashing trick, which maps several categories to the same row and therefore destroys the distinction between them. This paper's contribution is a way to cut the number of stored vectors **while still giving every category a unique representation**.

Methods and models used. The core idea is the **quotient-remainder trick**. For an item index i and a chosen divisor m , compute two indices, $i \bmod m$ and i divided by m , look up a vector in each of two smaller tables, and combine them. Every item gets a distinct pair, so uniqueness is preserved, while storage drops from one table of size $|S|$ to two tables of roughly $|S|/m$ and m . The paper generalises this to **complementary partitions** — any set of partitions whose intersection uniquely identifies each item — and tests several ways of combining the looked-up vectors, including element-wise multiplication and concatenation, plus a path-based variant using small neural networks.

Results. Tested on click-through rate prediction with the Kaggle Criteo Ad Display dataset, using DCN and Facebook's DLRM as the host architectures. The method reduced embedding table size substantially while staying close to the accuracy of the full tables. The authors' concrete recommendation is to **combine thresholding with the quotient-remainder trick**, and they report that the best combining operation depends on the network — element-wise multiplication worked best in their two cases. They found their path-based variant did **not** beat the simpler operation-based version.

Limitations. The setting is CTR prediction on tabular categorical features, not sequential recommendation, so the results do not transfer directly. Accuracy is traded away, and how much depends on the divisor and the combining operation, both of which need tuning. The motivation is server-side memory and training cost rather than fitting on a device, so there are no device measurements and no privacy analysis. Their own negative result on path-based embeddings is a useful caution that added complexity does not automatically help.

Relevance to Model 3. This is the published basis for the "share vectors between items" step in our compression plan, and it is the one technique that reduces the table below what quantisation can achieve. The paper also gives us a **defensible order of operations**: start with thresholding and quotient-remainder rather than anything more elaborate, and expect the

combining operation to need tuning. Their negative path-based result tells us not to reach for a neural composition function first.

Paper 3 — Stronger Privacy for Federated Collaborative Filtering With Implicit Feedback

Minto, Haller, Haddadi & Livshits (Brave Software, Imperial College London). **RecSys 2021**. DOI 10.1145/3460231.3474262

Summary. The privacy mechanism Model 3's update step needs. The authors note that most privacy-aware recommender work addresses explicit feedback such as ratings, while **implicit** feedback — views and clicks, which is what our data is — has received much less attention. They build a federated recommender for implicit data under **user-level local differential privacy**.

Methods and models used. Built on matrix-factorisation federated collaborative filtering. Each user's device computes gradient updates locally and privatises them **before** transmission, so the server never sees raw data. Two parameters control the trade-off: **epsilon**, the per-update privacy budget, and **k**, how many privatised gradient updates each user sends. They add a **proxy network** that anonymises and shuffles reports before they reach the recommender, which reduces what they call the fingerprinting surface — the server cannot link updates back to individuals even across rounds. The paper includes a proof that their algorithm satisfies epsilon-local differential privacy.

Results. Evaluated on MovieLens using Hit Ratio at 10. On a subset of 50,000 users and 5,000 items they reach **HR@10 of up to 0.68** at **epsilon = 2.5** with $k = 50$ and $k = 100$. On the full dataset they report **HR@10 above 0.5** while preserving the privacy guarantee. They acknowledge directly that the private system **does not match the performance of its non-private counterpart**, and they also report the communication cost of running the protocol.

Limitations. Matrix factorisation, not a sequential model, so it does not use interaction order at all — a significant gap relative to our architecture. Local DP requires far more noise than central DP for the same guarantee, which is why the accuracy cost is visible. The proxy network is an additional piece of infrastructure and relies on assumptions about the proxy not colluding with the recommender. Results are on MovieLens only, so behaviour on a sparse large-catalogue dataset is untested.

Relevance to Model 3. This gives us three concrete things. It is the **precedent for local DP on implicit feedback**, which is exactly our setting. Its **epsilon = 2.5** gives us a defensible reference point, so our own privacy budget is anchored in the literature rather than chosen arbitrarily. And its two-parameter structure, epsilon for per-update noise and k for how many updates are sent, maps directly onto the two things our design already planned to sweep: noise level and reporting frequency. It also confirms the honest expectation that a private version loses accuracy against a non-private one, which is what our own result will show.

What each paper actually did

Paper	Model	Data	Measured size	Measured privacy	Sequential
-------	-------	------	---------------	------------------	------------

RULE, KDD 2021	LightGCN	Amazon-Book, Yelp2020	Yes, 5/10/25 MB	No	No
Compositional, KDD 2020	DLRM, DCN	Criteo CTR	Yes	No	No
LDP Federated CF, RecSys 2021	Matrix factorisation	MovieLens	No	Yes, epsilon = 2.5	No
Model 3, ours	SASRec	MovieLens, Beauty	Yes	Yes	Yes

The gap we are filling

Reading the three together, the pattern is clear and it defines our contribution.

The compression papers measure size but not privacy. RULE and Compositional Embeddings both reduce memory carefully and report the accuracy cost precisely. Neither reports a privacy budget or runs an attack. RULE cites privacy as a reason to move computation on-device and then never measures it.

The privacy paper measures leakage but not size. Minto et al. provide a formal guarantee and a concrete epsilon, but the model is matrix factorisation and there is no memory budget or device constraint anywhere in the evaluation.

None of the three is a sequential model. All use either graph-based collaborative filtering, CTR prediction over tabular features, or matrix factorisation. None uses the order of a user's interactions, which is the basis of SASRec and of every modern sequential recommender.

Model 3 sits where all three constraints apply at once: a **sequential** recommender, under a **memory budget**, with a **measured privacy guarantee**, and with both costs reported against the same baseline on the same data. That combination is what the literature does not currently provide.

One consequence for our design. Because none of these papers combines size and privacy, there is no published number that tells us how the two costs interact — whether compressing a model makes the privacy noise hurt more, less, or the same. That interaction is a genuine open question and a reasonable thing for us to report, even if the answer turns out to be uninteresting.